جامعة الكويت
KUWAIT UNIVERSITY

CS223 Project Report

Name	ID
Ali Oudeh	2211114760
Naser Salamat	2192115811
Abbas Ahmadi	2221190136

4 May, 2026

Table of Content

Table of Content **2**

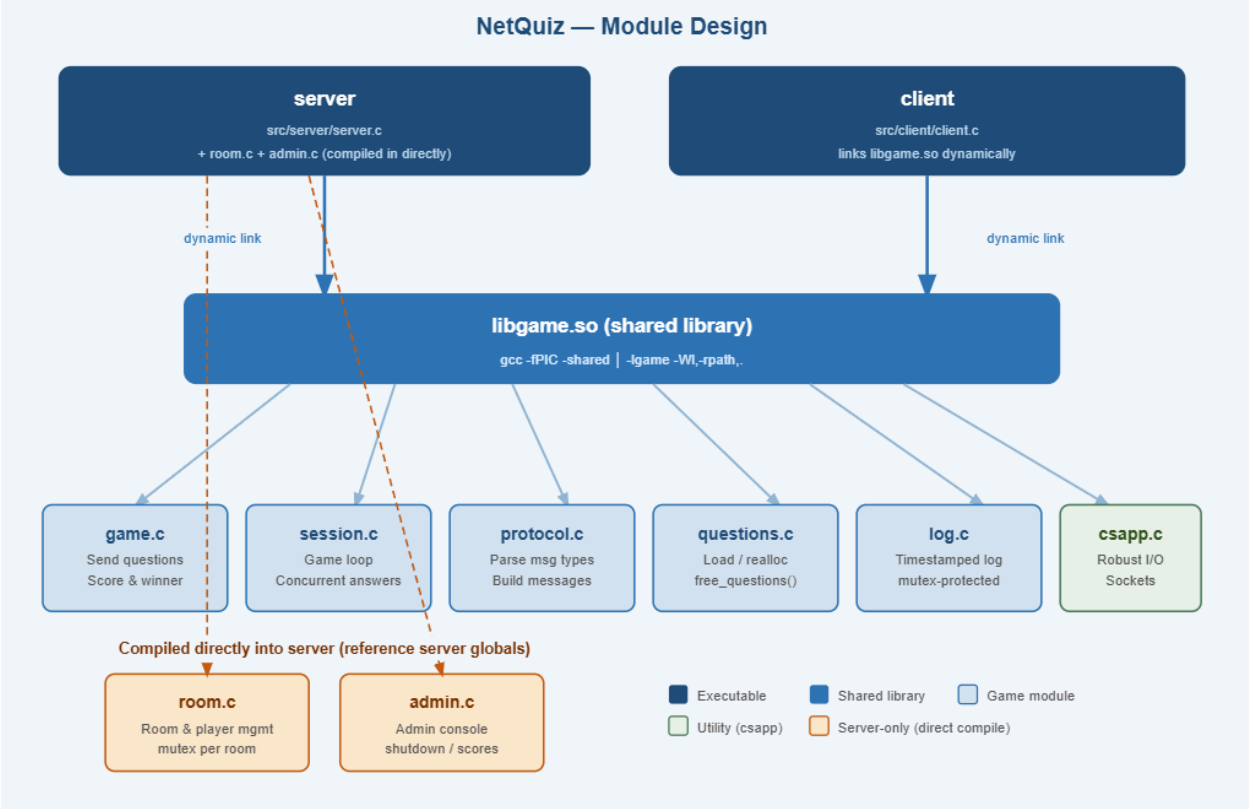
1. Overview 3
2. Module Design 3
3. Multithreading Model & Room Management 4
4. Synchronization Strategy & Race Condition 4
5. Memory Management 4
6. Testing Scenarios 5
7. Failure Handling & Shutdown Policy 5
8. Contribution Matrix 5

1. Overview

Netquiz is a multiplayer quiz game built fully in C, running over a TCP network, and it follows a general client-server architecture using **csapp.h,** where players connect from separate machines, compete by answering questions, and see a winner declared at the end of each game session.

2. Module Design

It describes how the games are split into focused C source files. Each handling one responsibility, and how the core game logic modules are compiled into a shared library **libgame.so** that they cover both server and client.



3. Multithreading Model & Room Management

NetQuiz uses **POSIX** threads [pthreads] to handle multiple players concurrently. The server creates a dedicated thread to accept incoming connections during the lobby and a monitor thread per player to detect disconnections. Once the game session starts, one answer-reder thread is created per player for each question round. Allowing all players to answer simultaneously rather than waiting for each other.

Each room operates independently with its own player list, current question, round state, scores, and final result. Rooms are stored in a global array (g_rooms[]) and can run concurrent game sessions without interfering with each other. A dedicated watcher thread per room manages the lobby countdown and triggers the game once enough players join. When a game ends, the room is reset under g_mtx and returned to a waiting state, ready to accept new players without requiring a server restart.

4. Synchronization Strategy & Race Condition

The server uses a two-level mutex design: g_mtx (global) protects lobby arrays and multi-room operations, while room->room_lock (per-room) protects player lists and game state. When both are needed, g_mtx is always acquired first to prevent deadlock.

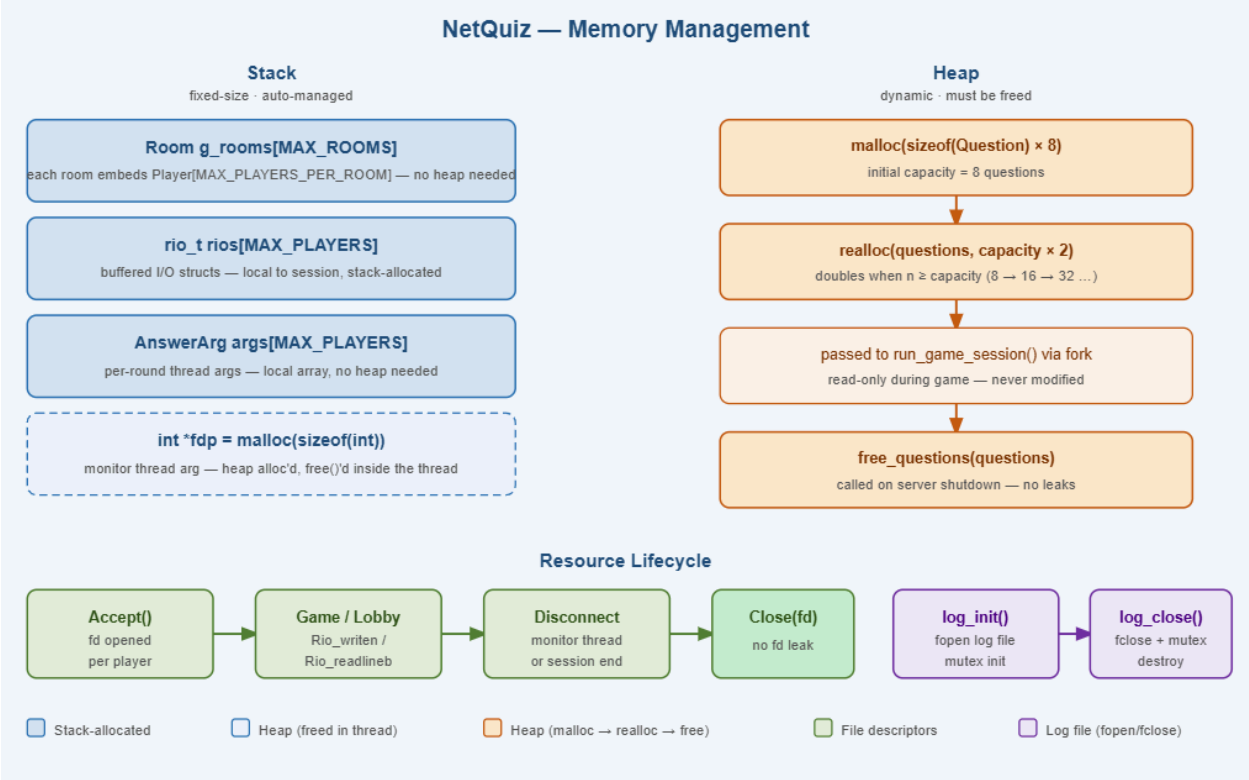
Critical sections are covered as follows: accepting players and room assignment use both locks; lobby countdown ticks use only g_mtx; disconnection handling, player snapshots, and admin commands use both locks; log writes use a separate log_mutex independent of all others.

Four main race conditions are addressed:
- Player joining while the lobby timer evaluates: both threads hold g_mtx.
- Player disconnecting during countdown: monitor thread resets lobby state under both locks.
- Two simultaneous connections to the same room: g_mtx is held across the full assignment block.
- Concurrent log writes: a private log_mutex in log.c serializes all log_event() calls.

Question timeouts (15s) are enforced inside update_score(), which holds room->room_lock during both the time check and the score write, preventing any race between the two.

5. Memory Management

NetQuiz handles dynamic allocation and deallocation throughout the program. The main area of dynamic allocation is the question array, which is loaded from **questions.txt** at startup using **malloc** and freed with **free_questions()** function when the server shuts down. Player data is stored in fixed-size stack arrays, avoiding heap allocation for per-player state. Also, the file descriptors are closed properly after each player disconnects or the game session ends, and that would prevent resource leaks.



6. Testing Scenarios

Scenario #1

A player disconnects from the lobby before the game starts, dropping the room below the minimum required players (2).
What will happen? The lobby countdown timer gets cancelled, and the remaining player receives a message: "Timer cancelled. Waiting for more players..."

Scenario #2

A player sends an invalid answer like Z or 5 instead of A, B, C, or D.
What will happen? The server rejects the answer and sends back: "Please choose the right letter," and the player's answer slot stays open until they reply with a valid choice or the timer runs out.

Scenario #3

A player disconnects mid-game while a question is active and other players are still answering.
What will happen? All remaining connected players receive a message that the player left, that player's slot is marked inactive, and the answer counter decrements, so the round can still finish without waiting for the dropped player.

Scenario #4

Both rooms (Room 0 and Room 1) are full (4 players each, 8 total), and a 9th player tries to connect
What will happen? The server has no available room to assign to the new player. The connection is rejected since all_rooms_full() returns true and the player cannot join.

Scenario #5

Two players finish a full game with the same score [highest score]
What will happen? After the last question, the server calls announce_winner(), broadcasts both of them as winners.

Scenario #6

The admin types **Ctrl+C** to try to stop the server while a game is running
What will happen? The server ignores the signal entirely. A message prints: "Ctrl+C ignored. Type: shutdown confirm". The server keeps running, and no players are affected. The admin should write “shutdown confirm” to stop the server.

Scenario #7

Room 0 fills up to its 4-player max, and a new player connects to the server

What will happen? The server's accept_fn loops through rooms in order, skips Room 0 (full), and assigns the new player to Room 1. The player gets placed there without any error, and Room 1 begins its own independent lobby countdown once it reaches the minimum player count (2).

7. Failure Handling & Shutdown Policy

Handling failures in the games is as important as implementing the game itself, because it helps debug any issues that might occur later. Client disconnection is handled by a per-player monitor thread blocking on recv(). When the socket closes, the thread removes the player under both locks, resets the lobby timer if needed, and notifies remaining players. No restart is required.

Other failure cases: a full server sends an **INFO** message and closes the connection under g_mtx; malformed JOIN messages cause the socket to close and loop back; a missing questions file causes immediate exit before any socket is opened.

SIGINT (Ctrl+C) is caught, but only prints a reminder using **write()**. The server keeps running. The only valid shutdown is the admin command "**shutdown confirm**", which notifies all clients, flushes the log, and calls _exit(0).

All events are logged to server.log with timestamps for a full audit trail across sessions.

8. Contribution Matrix

Team Member	**Precentage**	**Topics**
Ali Oudeh	35%	Multithreading - Colors Bonus
Naser Salamat	30%	Multi-Rooms with the bonus [more than 2 rooms] - Game Logic
Abbas Ahmadi	35%	Server Logs - Admin Commands - Questions Timeout